UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA
DEPARTAMENTO DE INFORMÁTICA · MAGÍSTER EN TECNOLOGÍAS DE LA INFORMACIÓN

Agentes autónomos basados en LLMs para el diagnóstico de incidentes en sistemas distribuidos: evaluación empírica comparativa y efecto del acceso a código y observabilidad

Kendru Estrada Rodríguez
Magíster en Tecnologías de la Información, Universidad Técnica Federico Santa María
Organización vinculada: Apside Ltda., Santiago, Chile
kendru@apside.cl
Profesor guía: Ricardo Ñanculef, Iván Pizarro

Resumen:

El diagnóstico de un incidente en un sistema distribuido requiere correlacionar información dispersa entre: código, logs y configuración de la infraestructura. Este estudio evaluó si un agente basado en modelos de lenguaje de gran tamaño, desarrollado por un equipo pequeño, resulta realmente útil para esta tarea, y cómo se compara con agentes comerciales ofrecidos por proveedores globales. Creamos "BugWizard", un agente que indexa el código y los registros de un proyecto, y genera respuestas mediante la técnica RAG (Retrieval-Augmented Generation). Posteriormente, lo comparamos con Claude Code y Gemini CLI a lo largo de tres fases. En la primera, dos expertos realizaron una evaluación ciega de los diagnósticos generados por ocho configuraciones distintas, empleando diecinueve escenarios sintéticos y diez incidentes reales. En la segunda fase, los sistemas debieron localizar fallos en 36 programas del conjunto de datos público ConDefects; para ello, generamos registros mediante la ejecución de los casos de prueba asociados. En la tercera fase, cuatro desarrolladores abordaron dieciséis tickets de soporte con límite de tiempo, alternando entre el uso de BugWizard y su flujo de trabajo habitual. Los resultados indican que el acceso a logs aporta la mejora más significativa al proceso de diagnóstico; en el caso de incidentes reales, la tasa de éxito en la identificación de la causa raíz aumentó del 40 % al 100 %. En la primera fase, BugWizard alcanzó un nivel de calidad equivalente al del mejor agente comercial; en la evaluación con ConDefects, su desempeño estuvo a la par de los sistemas comerciales (logrando una tasa de éxito del 78 % en la primera línea de código sugerida); además, redujo el tiempo de diagnóstico de los desarrolladores en aproximadamente un 70 %, superando la reducción del 30 % que se había planteado como hipótesis inicial.

Palabras clave: modelos de lenguaje de gran escala; agentes autónomos; RAG; análisis de causa raíz; localización de fallas; observabilidad; ingeniería de software.

1. Introducción

Apside Ltda. es una consultora chilena de desarrollo de software que presta servicios a clientes de los sectores bancario, seguros y minorista. Sus equipos operan sistemas distribuidos compuestos por microservicios y funciones serverless alojados en nubes públicas. Cuando surge un incidente en el entorno de producción, el ingeniero de turno debe reconstruir la secuencia de los hechos a partir de fuentes dispersas: el código se encuentra en un repositorio, los logs de ejecución residen en un servicio usado para este proposito, la configuración está definida en plantillas de infraestructura y el contexto de negocio se halla en tickets anteriores. Una parte considerable del tiempo dedicado al diagnóstico se consume recopilando estos elementos, en lugar de analizar y comprender el fallo en sí.

La oferta de agentes basados en LLM's para programación creció rápidamente entre 2024 y 2026; sin embargo, las empresas de tamaño mediano carecen de criterios claros para realizar una elección de un agente en particular. Los análisis que se han hecho provienen, en su mayor parte, de grandes proveedores de servicios en la nube (que se basaron en miles de sus propios incidentes y ajustaron sus modelos utilizando sus propios datos), un contexto que no resulta aplicable a una firma de consultoría con equipos de entre diez y treinta personas. Este estudio busca obtener evidencia que sea pertinente para ese entorno específico.

Con este fin, se creó BugWizard, que es un agente desarrollado en Apside por un equipo reducido, y se comparó con dos agentes comerciales de propósito general: Claude Code y Gemini CLI. La comparación consistió en variar sistemáticamente la información proporcionada a cada sistema (solo código; código más logs; y código, logs y el relato de un desarrollador) para determinar qué factor explicaba las diferencias en el rendimiento. La validación se llevó a cabo en tres fases, cada una de las cuales comprendía un experimento distinto: una evaluación a ciegas realizada por expertos mediante un conjunto interno de casos de prueba, una validación externa utilizando el conjunto de datos público ConDefects y un experimento con desarrolladores sujeto a restricciones de tiempo.

La **hipótesis principal (H1)** sostiene que BugWizard, al integrarse en el flujo de trabajo del desarrollador, reduce el tiempo de diagnóstico en al menos un 30 % sin comprometer la precisión. Además, se plantean 3 hipótesis secundarias: que las configuraciones con acceso a logs ofrecen mejores resultados de diagnóstico que aquellas limitadas a la visualización del código (H2); que el rendimiento relativo observado en un conjunto de pruebas interno se mantiene en un *benchmark* público (H3); y que un agente desarrollado mediante RAG por un equipo pequeño puede lograr resultados comparables a los de agentes comerciales de proveedores globales para esta tarea específica (H4). Esto último tiene una relevancia práctica: de confirmarse, una empresa podría optar por una herramienta propia (sobre la cual tiene un mayor control) sin sacrificar la calidad.

El informe se organiza de la siguiente manera: el capítulo 2 presenta los conceptos, el estado del arte y la brecha que este trabajo aborda; el capítulo 3 describe BugWizard y los agentes utilizados para la comparación; el capítulo 4 expone las tres fases de validación, incluyendo en cada una su diseño, resultados y observaciones; el capítulo 5 analiza los hallazgos en su conjunto, junto con sus limitaciones; y el capítulo 6 extrae conclusiones en relación con las hipótesis. Los detalles estadísticos se han incluido en los apéndices para facilitar la lectura del cuerpo principal del texto.

2. Marco teórico y estado del arte

2.1. Conceptos base

Modelos de lenguaje de gran escala (LLM). Se trata de redes neuronales con una arquitectura de tipo "transformer" [1], entrenadas con grandes volúmenes de texto para predecir la siguiente palabra. Su propiedad más útil en este contexto es el aprendizaje en contexto: pueden resolver una tarea nueva si se les proporciona la información necesaria dentro del texto de entrada, sin necesidad de un reentrenamiento [2]. Además, razonan de manera más eficaz cuando se les pide que descompongan un problema en pasos intermedios, una técnica conocida como "chain of thought" [3]. Esto las hace adecuadas para empresas que no disponen de datos suficientes para entrenar sus propios modelos.

Agentes autónomos basados en LLM. Un agente utiliza un LLM como controlador, complementándolo con capacidades de planificación, memoria y herramientas externas, tales como la lectura de archivos, la ejecución de comandos o la consulta a servicios [4], [5]. Anthropic distingue entre "flujos de trabajo", en los que los pasos están predefinidos por el programador, y "agentes", en los que el modelo decide qué herramienta utilizar en cada paso a partir del resultado obtenido en el paso anterior [6]. Esta dinámica de decisión y

ejecución puede describirse mediante el patrón ReAct, que articula el razonamiento del modelo con la ejecución de acciones y la observación de sus resultados [7].

Agentes comerciales de programación. Claude Code, de Anthropic, y Gemini CLI, de Google, son agentes que operan en un repositorio local a través de la terminal: leen y realizan búsquedas en archivos, ejecutan comandos y pueden conectarse a servicios externos [8], [9]. Representan el estado actual de la práctica industrial y se evalúan de normalmente mediante benchmarks como SWE-bench, el cual mide si un agente es capaz de resolver problemas del mundo real en repositorios públicos [10]. En este trabajo, sirven como referencia de "agentes de clase mundial" frente a los cuales se compara BugWizard.

.

Generación aumentada por recuperación (RAG). RAG combina un motor de búsqueda con un LLM: los documentos se transforman en vectores numéricos, se recuperan los segmentos más similares a la consulta y estos se proporcionan al modelo junto con la consulta misma [11]. De este modo, el modelo genera respuestas basadas en información específica y actualizada, lo que reduce las alucinaciones. Estos vectores se almacenan en un índice especializado; BugWizard utiliza FAISS, una biblioteca para la búsqueda basada en la similitud [12]. La aplicación de RAG en el soporte de TI, mediante la indexación de tickets anteriores y guías de resolución, fue documentada por Toro Isaza et al. [13].

Protocolo de contexto de modelo (MCP). Es un protocolo abierto, propuesto en 2024, que estandariza la forma en que un agente se conecta a fuentes de datos y herramientas externas [14]. Varios agentes pueden utilizar un mismo servidor MCP que accede a logs, sin necesidad de hacer adaptaciones. En este trabajo, Claude Code y Gemini CLI accedieron a los logs de AWS (CloudWatch) a través de un servidor MCP.

AIOps y análisis de causa raíz. AIOps es la aplicación de análisis de datos y aprendizaje automático a la operación de sistemas de TI [15]. Dentro de AIOps, el análisis de causa raíz busca el origen de un incidente cruzando logs, métricas y eventos; Wang y Qi resumen las técnicas del área [16]. Los LLM agregan la capacidad de leer información no estructurada, como mensajes de error, trazas de excepción o comentarios en el código.

Localización de fallas y métricas Top-N. Localizar una falla es señalar la línea o el bloque de código responsable de un comportamiento incorrecto [17]. Se mide con métricas Top-N: el sistema propone una lista de líneas ordenada por confianza, y se cuenta un acierto si la línea defectuosa aparece entre las primeras N. Top-1 mide si la primera hipótesis es la correcta; Top-3 y Top-5 se acercan más al uso real, porque revisar tres líneas candidatas es un costo aceptable.

Tiempo medio de diagnóstico (MTTD). Es el tiempo entre que se detecta un incidente y se identifica su causa raíz [18]. Se diferencia del tiempo medio de resolución (MTTR), que incluye además corregir y desplegar. Se eligió el MTTD porque mide justo la parte del trabajo en la que un agente de diagnóstico puede ayudar, y deja fuera aprobaciones y despliegues, que no dependen de la herramienta.

2.2. Estado del arte

Ahmed et al., Microsoft Research (ICSE 2023). Primer estudio sistemático sobre LLM para diagnosticar incidentes de producción, con más de 40.000 incidentes de más de mil servicios en la nube [19]. Mostró que los modelos pueden proponer causas raíz y mitigaciones útiles, y dejó dos desafíos abiertos que este trabajo retoma: el conocimiento del modelo envejece y el enunciado del incidente no basta; hace falta contexto adicional como logs, métricas o dependencias.

Meta (2024). Sistema de análisis de causa raíz sobre el repositorio web de Meta que combina búsqueda heurística por propiedad del código y grafos de dependencias con un modelo Llama 2 de 7.000 millones de parámetros, ajustado con investigaciones anteriores [20]. Acierta en 42% de los casos al poner la causa entre

los cinco cambios sugeridos. El ajuste con datos internos fue clave, algo que una empresa del tamaño de Apside no puede replicar.

Triangle, Microsoft Azure (2025). Sistema de varios agentes para clasificar incidentes en Azure, donde cada agente representa el conocimiento de un equipo [21]. Muestra que especializar el contexto, más que el modelo, es donde está la ganancia.

Roy et al. (2024). Evaluación de un agente ReAct con herramientas de búsqueda sobre incidentes de producción de una gran empresa [22]. Aporta evidencia de que los agentes funcionan sin ajuste fino y una clasificación de sus modos de falla.

RCACopilot (EuroSys 2024) y LogSage (2025). Chen et al. reportan hasta 0,766 de precisión al clasificar causas raíz sobre un año de incidentes de un servicio de Microsoft, usando cadena de pensamiento y recolección automática de información [23]. LogSage usa LLM para detectar y corregir fallas en procesos de integración y despliegue continuo, validado en ByteDance [24].

Benchmarks públicos. OpenRCA reúne 335 fallas de tres sistemas empresariales con más de 68 GB de logs, métricas y trazas [25]; Traversal compara modelos de última generación en análisis de causa raíz y concluye que razonar más no siempre mejora el resultado y sí aumenta la latencia [26]. En el otro extremo, ConDefects reúne programas defectuosos de concursos de programación, con la línea del defecto y sus casos de prueba, pensado para evaluar localización de fallas sin contaminación de los datos de entrenamiento [27].

2.3. Vacío identificado

La revisión deja cuatro vacíos.

Primero, la literatura evalúa código o logs, pero casi nunca ambos de forma controlada. Los benchmarks de localización de fallas, como ConDefects, entregan solo el código [27]. Los de AIOps, como OpenRCA, entregan telemetría, pero no el código del servicio [25]. Los sistemas industriales que usan ambas fuentes [19], [20] no miden cuánto aporta cada una. No se encontró una medición controlada de cuánto mejora el diagnóstico al agregar logs a una base de solo código.

Segundo, se mide la precisión del modelo, pero no el efecto sobre el trabajo de las personas. Ninguno de los estudios analizados mide en qué medida se reduce el tiempo real que un desarrollador dedica al diagnóstico. Una herramienta podría ofrecer la respuesta correcta en el 80 % de los casos y, aun así, no ahorrar nada de tiempo si el ingeniero tiene que verificar cada respuesta desde cero.

Tercero, el contexto organizacional es distinto. Los resultados de Microsoft y Meta se obtuvieron con miles de incidentes propios y modelos ajustados. Una consultora mediana no tiene esos datos ni ese presupuesto, y aun así debe elegir entre herramientas comerciales o construir algo propio.

Cuarto, no hay evidencia sobre si un agente construido por un equipo pequeño compite con los agentes comerciales. Los trabajos industriales describen sistemas de grandes empresas. Falta saber si una solución sencilla, basada en RAG y modelos disponibles como servicio, puede acercarse a los agentes de propósito general de los proveedores globales.

3. Solución desarrollada y sistemas comparados

3.1. Qué hace BugWizard

BugWizard es una plataforma web desarrollada en Apside para apoyar el diagnóstico de incidentes. Su uso tiene dos momentos:

- **Configuración del proyecto (una sola vez).** El equipo registra el proyecto: los repositorios de código (GitHub o Bitbucket), las fuentes de logs (AWS CloudWatch, New Relic, Rancher o archivos locales) y las credenciales de acceso, que quedan cifradas.
- **Diagnóstico de un ticket.** Al crear una conversación para un incidente, BugWizard descarga los logs de la ventana de tiempo indicada y los indexa junto con el código y el contexto del proyecto. Luego el usuario describe el problema; BugWizard recupera los fragmentos de código y de logs más relacionados con la pregunta y hace **una sola llamada al LLM** con ese material. La respuesta incluye la causa probable, los archivos involucrados y una propuesta de corrección.

En los términos del capítulo 2, la versión evaluada de BugWizard es un sistema **RAG de una llamada**: no usa herramientas ni itera, y los logs que ve son los que se descargaron e indexaron al crear la conversación (acceso *estático*). Esto lo diferencia de Claude Code y Gemini CLI, que consultan los logs por su cuenta las veces que estimen necesario (acceso *interactivo*).

3.2. Arquitectura

La Figura 1 muestra la arquitectura desplegada en una cuenta de AWS. El usuario entra desde el navegador a la aplicación web (Quasar/Vue), servida desde un bucket privado de S3 a través de CloudFront, y se autentica con Amazon Cognito. Las solicitudes al backend pasan por una segunda distribución de CloudFront y un balanceador de carga hasta el servicio principal, que corre en contenedores ECS Fargate.

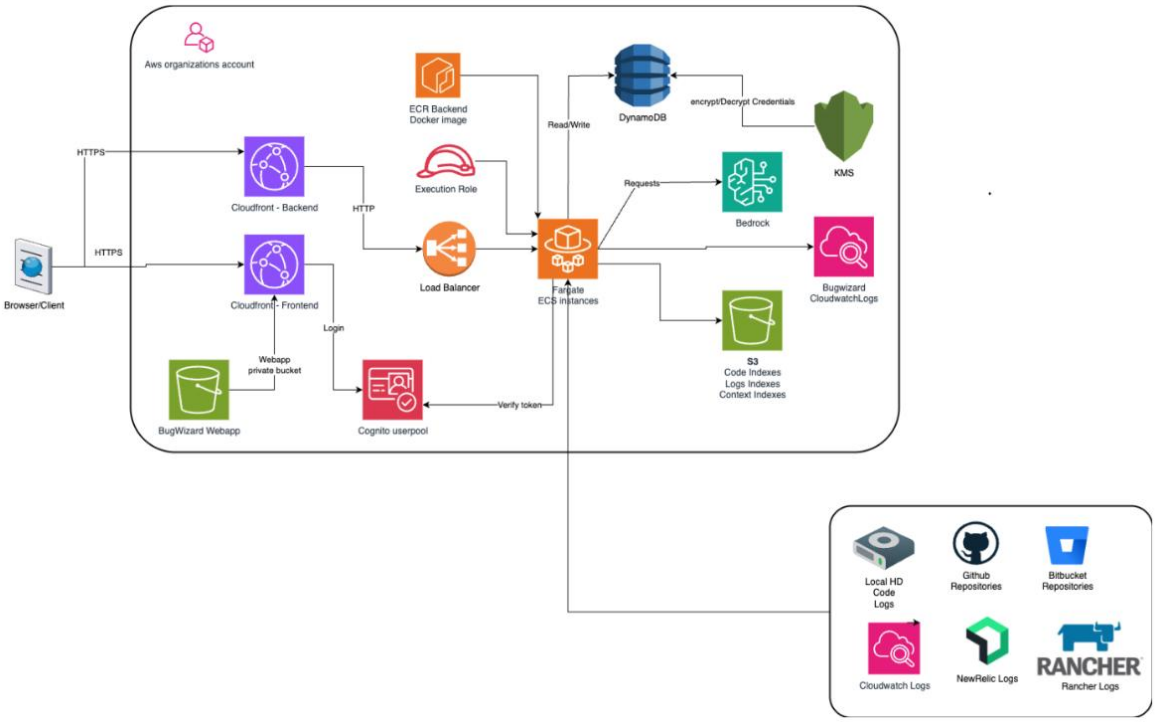


Figura 1. Arquitectura de BugWizard en AWS. Abajo a la derecha, las fuentes externas de código y de logs que la plataforma puede indexar.

El servicio principal (*ms-agente-tickets*) está escrito en NestJS y usa LangChain para orquestar el modelo. Sus dependencias son:

- **Amazon Bedrock**, que entrega el LLM. Usar un servicio de inferencia dentro de la cuenta corporativa responde a un requisito de los clientes financieros: los datos no salen del perímetro de la empresa. El modelo se cambia por configuración.

- **Amazon S3**, donde se guardan los índices vectoriales FAISS del código, de los logs y del contexto de cada proyecto.
- **DynamoDB**, donde se guardan proyectos y conversaciones; las credenciales se cifran con **AWS KMS**.
- **Fuentes externas**: repositorios GitHub y Bitbucket, y logs de CloudWatch, New Relic, Rancher o disco local.

Que la configuración del proyecto quede guardada parece un detalle, pero tuvo consecuencias importantes en el experimento con desarrolladores (sección 4.4).

3.3. Sistemas comparados

La Tabla 1 resume las diferencias entre los tres sistemas en la forma en que se usaron en las pruebas.

	BugWizard	Claude Code	Gemini CLI
Origen	Desarrollo propio (equipo pequeño)	Anthropic	Google
Estrategia	RAG, una llamada al modelo	Agente con herramientas (iterativo)	Agente con herramientas (iterativo)
Acceso al código	Índice vectorial del repositorio	Lectura directa de archivos	Lectura directa de archivos
Acceso a logs	Estático: descargados e indexados al crear la conversación	Interactivo: servidor MCP de CloudWatch	Interactivo: servidor MCP de CloudWatch
Dónde corre el modelo	Bedrock, dentro de la cuenta corporativa	Bedrock (en las pruebas)	API de Google

Tabla 1. Sistemas comparados, tal como se configuraron en las pruebas.

El servidor MCP usado por Claude Code y Gemini CLI fue *Log Analyzer with MCP*, de AWS Labs [28], que ofrece herramientas para listar grupos de logs, buscar y filtrar eventos, resumir actividad, encontrar patrones de error y correlacionar logs de varios servicios.

Los modelos usados en cada fase se indican en el diseño de cada una. En la Fase 1 BugWizard y Claude Code usaron Claude Sonnet 4; en las Fases 2 y 3, Claude Sonnet 5, el modelo vigente en Bedrock al momento de ejecutarlas. Gemini usó el modelo flash disponible en cada fecha.

4. Validación

4.1. Estrategia general e hipótesis

La validación combinó tres tipos distintos de experimentos, cuyas fortalezas se complementan entre sí. La fase 1 emplea escenarios realistas evaluados por expertos, aunque estos carecen de una única respuesta objetiva. La fase 2 utiliza un banco de pruebas público con respuestas verificables, si bien se basa en programas más sencillos que los que se encontrarían en un sistema del mundo real. La fase 3 mide el impacto en el trabajo de personas reales, aunque con un número limitado de participantes. El hecho de que los tres apunten en la misma dirección es lo que sustenta las conclusiones. Cada fase se presenta junto con su diseño, sus resultados y sus observaciones.

	Enunciado	Fase
H1	BugWizard, integrado al flujo del desarrollador, reduce el tiempo de diagnóstico (MTTD) en al menos 30% sin empeorar la precisión de la causa raíz.	3
H2	Las configuraciones con acceso a logs producen mejores diagnósticos que las que solo disponen del código, según evaluación ciega de expertos.	1
H3	El orden de desempeño entre sistemas observado en el banco interno se mantiene en un benchmark público independiente.	1 y 2
H4	Un agente construido con RAG por un equipo pequeño (BugWizard) logra un desempeño comparable al de agentes comerciales de proveedores globales (Claude Code, Gemini CLI) en esta tarea.	1 y 2

Tabla 2. Hipótesis y fase que las contrasta.

4.2. Fase 1: evaluación ciega sobre el banco interno

4.2.1. Diseño del experimento

Casos. Se recopiló/construyó un conjunto de datos compuesto por 30 casos. 20 de ellos son escenarios sintéticos, a saber: aplicaciones serverless en AWS (Lambda, API Gateway, DynamoDB, SQS, EventBridge, Step Functions), creadas específicamente para este estudio, cada uno de los cuales contiene un defecto conocido. Para garantizar la existencia de registros (*logs*) reales, cada aplicación se sometió a pruebas periódicas mediante *canaries* de CloudWatch Synthetics, lo que generó tráfico y, por lo tanto, errores dentro de CloudWatch. Los 10 casos restantes corresponden a incidentes reales experimentados por clientes de Apside; estos han sido anonimizados y sus resoluciones, documentadas (Tabla 3).

.

ID	Incidente real	Complejidad	ID	Incidente real	Complejidad
T001	NullPointerException en creación de usuario	Baja	T006	Fuga de memoria en servicio de caché	Muy alta
T002	Consulta SQL sin índice	Baja	T007	Error de serialización JSON	Media
T003	Deadlock entre transacciones	Alta	T008	Permisos IAM insuficientes sobre S3	Alta
T004	Error OAuth2 por token expirado	Media	T009	Validación inconsistente frontend/backend	Media
T005	Variables de entorno incorrectas	Alta	T010	Pérdida de eventos asíncronos	Muy alta

Tabla 3. Incidentes reales del banco de pruebas.

Configuraciones. Cada caso se procesó con ocho configuraciones (Tabla 4). En Claude Code y Gemini CLI se variaron tres niveles de información; en BugWizard, dos versiones de la pregunta.

Configuración	Qué recibió
Claude Code / Gemini CLI — **solo código**	El repositorio y una instrucción genérica: "identifica la causa raíz de un error en el código fuente". Sin logs.
Claude Code / Gemini CLI — **código + logs**	El repositorio, el síntoma reportado por el usuario y acceso a los logs de CloudWatch mediante el servidor MCP, que el agente consulta por su cuenta.
Claude Code / Gemini CLI — **detallado**	Lo mismo que *código + logs*, más un relato del desarrollador que explica cómo debería funcionar el flujo. Es la única diferencia con la configuración anterior.
BugWizard — **prompt 1**	Código y logs indexados (acceso estático) y el síntoma reportado. Una sola llamada al modelo.
BugWizard — **prompt 2**	Lo mismo que *prompt 1*, más el mismo relato del desarrollador que recibe la configuración *detallado*.

Tabla 4. Configuraciones de la Fase 1. *Prompt 1* equivale en información a *código + logs*, y *prompt 2* a *detallado*.

El recuadro siguiente muestra la diferencia entre *prompt 1* y *prompt 2* para el escenario 004, una suscripción que queda activa sin que el usuario reciba los permisos premium.

Prompt 1 (síntoma): "El usuario ve su suscripción activa en el dashboard pero no puede acceder a funciones premium."

Prompt 2 (síntoma + relato): "El usuario ve su suscripción activa en el dashboard pero no puede acceder a funciones premium. La función debería funcionar como sigue: guardar la suscripción en DynamoDB; inmediatamente después llamar a una función que cambia los permisos en Cognito (si esto falla, está definido un mecanismo de reintentos)."

Todas las configuraciones se ejecutaron en febrero de 2026. Claude Code y BugWizard usaron Claude Sonnet 4 (vía Bedrock) y Gemini CLI usó gemini-2.5-flash.

Evaluación. Las 8 respuestas de cada caso se anonimizaron, se desordenaron y se entregaron a **dos expertos** que conocían el incidente y su solución: el autor y otro ingeniero de Apside. Cada experto calificó de 1 a 5 la calidad general, la explicación, las propuestas de solución y la ausencia de información inventada (alucinación), y marcó si la respuesta identificó la causa raíz (sí/no). La rúbrica está en el Anexo A.

4.2.2. Resultados en escenarios sintéticos

Se analizaron 19 de los 20 escenarios (el T017 no tuvo evaluaciones), con 288 calificaciones en total.

Configuración	General	Explicación	Propuestas	Sin aluc.	Identificó
Claude Code (detallado)	**4,19**	4,19	4,00	4,38	92%
Claude Code (código + logs)	4,11	4,14	3,86	4,34	89%
BugWizard (prompt 2)	4,11	4,14	3,97	3,97	83%
BugWizard (prompt 1)	4,08	4,25	4,03	4,08	86%
Gemini CLI (código + logs)	3,61	3,53	3,36	4,22	86%
Gemini CLI (detallado)	3,53	3,25	3,28	4,31	86%
Claude Code (solo código)	3,25	3,31	2,86	3,56	58%
Gemini CLI (solo código)	3,08	3,03	3,00	3,64	61%

Tabla 5. Escenarios sintéticos (n=19): promedio de las calificaciones (1 a 5) y porcentaje de respuestas que identificaron la causa raíz.

El análisis estadístico (Anexo B) muestra que casi todas las configuraciones encuentran una causa plausible, y que la diferencia está en la calidad de la explicación y de la solución propuesta. En los escenarios sintéticos,

que tienen un solo defecto bien delimitado, identificar el error es relativamente fácil; lo difícil es explicarlo bien y proponer algo útil.

Se distinguen tres grupos. Arriba están las cuatro configuraciones de Claude Code y BugWizard con logs, con calificaciones entre 4,08 y 4,19, **sin diferencias estadísticas entre ellas**. En el medio, Gemini CLI con logs. Abajo, las dos configuraciones de solo código.

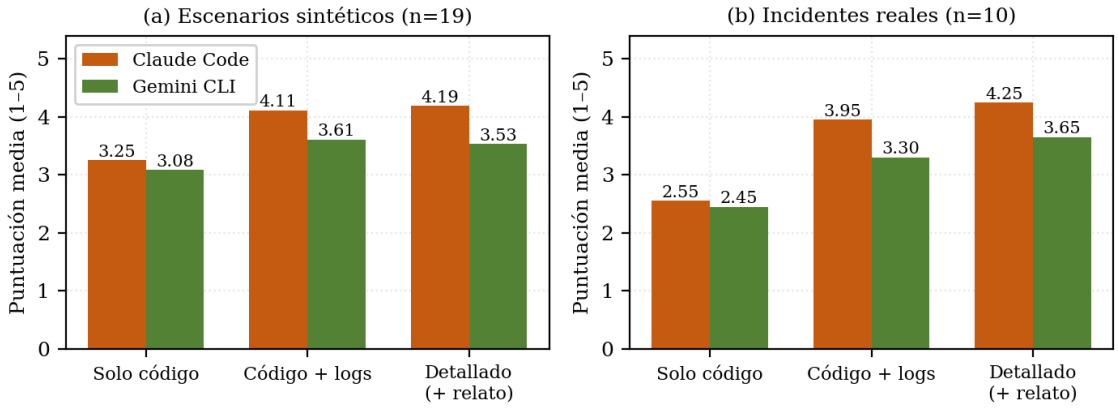


Figura 2. Calificación promedio según la información disponible, por sistema y tipo de caso.

La Figura 2(a) muestra el efecto de la información: pasar de *solo código* a *código + logs* sube a Claude Code de 3,25 a 4,11 y a Gemini CLI de 3,08 a 3,61. Pasar de *código + logs* a *detallado* casi no cambia nada (+0,08 en Claude, −0,08 en Gemini). Como entre esas dos configuraciones solo cambia el relato del desarrollador, la conclusión es que **el relato agrega poco cuando el agente ya tiene acceso a los logs**. Lo mismo ocurre en BugWizard entre *prompt 1* y *prompt 2*.

4.2.3. Resultados en incidentes reales

Los diez incidentes reales produjeron 160 calificaciones. Como son sistemas en producción con más servicios involucrados, las diferencias se acentúan.

Configuración	General	Explicación	Propuestas	Sin aluc.	Identificó
Claude Code (detallado)	**4,25**	4,85	4,45	5,00	100%
BugWizard (prompt 2)	4,05	4,65	4,00	5,00	100%
Claude Code (código + logs)	3,95	4,60	3,95	5,00	100%
BugWizard (prompt 1)	3,70	4,05	3,80	5,00	100%
Gemini CLI (detallado)	3,65	4,00	4,00	5,00	75%
Gemini CLI (código + logs)	3,30	3,65	3,05	5,00	40%
Claude Code (solo código)	2,55	3,00	2,55	4,00	40%
Gemini CLI (solo código)	2,45	2,55	2,15	4,00	10%

Tabla 6. Incidentes reales (n=10): promedio de las calificaciones y porcentaje de respuestas que identificaron la causa raíz.

El resultado más claro está en la última columna. Con solo código, Claude Code identifica la causa raíz en 4 de 10 incidentes y Gemini CLI en 1 de 10. Con logs, las cuatro configuraciones de Claude Code y BugWizard

llegan a 10 de 10. La razón es que en sistemas serverless la causa muchas veces no está en el código sino en cómo el código interactúa con su entorno: permisos IAM mal configurados, variables de entorno incorrectas o eventos que se pierden entre servicios. Tres incidentes (T005, T008 y T010) no pueden diagnosticarse sin evidencia de ejecución, y ninguna configuración sin logs los resolvió.

El efecto de los logs también es mayor que en los escenarios sintéticos: en Claude Code la ganancia pasa de 0,86 a 1,40 puntos (Figura 2(b)). Los escenarios construidos, con un defecto único, subestiman el valor de los logs.

En la dimensión de alucinación, todas las configuraciones con logs obtuvieron 5,00 y las de solo código 4,00. Un agente que puede citar una línea de log concreta inventa menos que uno que debe adivinar a partir del código.

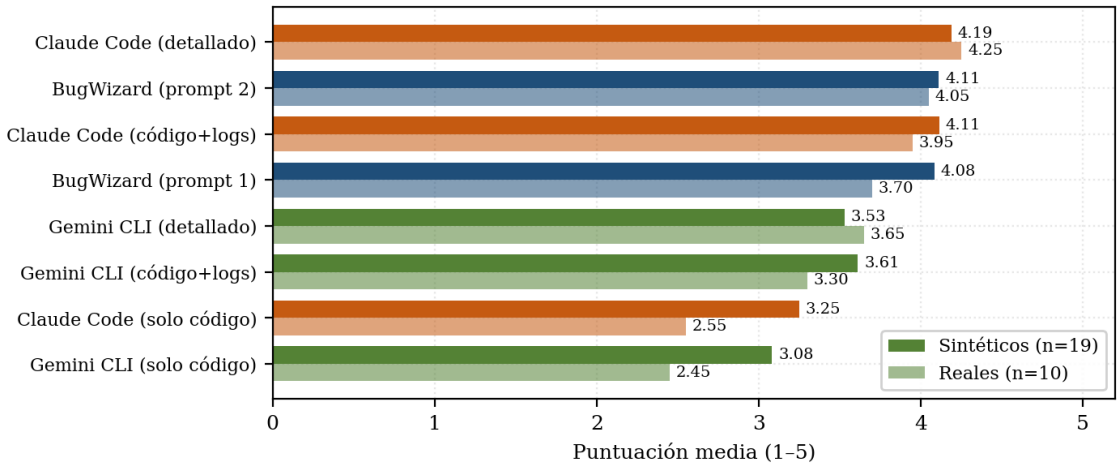


Figura 3. Ranking de las ocho configuraciones en ambos conjuntos de casos. El orden se mantiene entre escenarios sintéticos e incidentes reales.

Acuerdo entre los expertos. Los dos expertos coincidieron en el orden de las configuraciones: produjeron exactamente las mismas cuatro primeras y las mismas dos últimas. Sí difirieron en la escala: uno calificó sistemáticamente más bajo que el otro, sin alterar el orden. Por eso el ranking es confiable, aunque las notas absolutas deban leerse con cautela (detalle en el Anexo B).

4.2.4. Observaciones de la Fase 1

En pocas palabras. El acceso a logs es lo que más mejora el diagnóstico, y su efecto crece con el realismo del caso: en incidentes reales llevó la identificación de la causa raíz de 40% a 100%. El relato del desarrollador agrega poco una vez que el agente tiene logs. BugWizard, con una sola llamada y logs indexados, quedó estadísticamente empatado con Claude Code, que consulta los logs de forma interactiva, y por encima de Gemini CLI.

Dos observaciones adicionales. Primero, el empate entre BugWizard y Claude Code es relevante porque ambos usaron el mismo modelo (Sonnet 4): con igual modelo, un RAG de una llamada con logs preindexados rindió lo mismo que un agente que explora los logs por su cuenta. Segundo, la configuración *solo código* de los escenarios sintéticos recibió solo la instrucción genérica, sin el síntoma reportado, de modo que su diferencia con *código + logs* mezcla el efecto de los logs con el de conocer el síntoma (se discute en la sección 5.5).

4.3. Fase 2: validación externa sobre ConDefects

4.3.1. Diseño del experimento

¿Por qué ConDefects? benchmark utilizado en la Fase 1 fue desarrollado por el autor y evaluado por expertos, lo que deja margen para el sesgo. Para abordar este problema, se empleó un conjunto de referencia público con un “ground truth" objetivo: ConDefects [27]. Este conjunto de datos consta de programas en Java y Python que contienen errores, los cuales fueron enviados a competiciones de AtCoder. Para cada programa, se proporcionan la línea que contiene el error, la versión corregida y los casos de prueba originales de la competición. Dado que se trata de problemas recientes, se reduce el riesgo de que los modelos los hayan encontrado durante su entrenamiento..

Muestra. Se eligieron 36 programas Python con una sola línea defectuosa, estratificados según la letra del problema en el concurso, que indica su dificultad: 14 fáciles (letras a y b), 8 medios (c y d) y 14 difíciles (e a h). La composición está en el Anexo C.

Logs construidos a partir de los casos de prueba. ConDefects no proporciona logs, pero sí los casos de prueba de cada problema, con la entrada y la salida esperada. Como BugWizard está pensado para trabajar con logs, se generó uno por programa simulando lo que registraría un sistema en ejecución:

- Se ejecutó el programa defectuoso contra todos sus casos de prueba.
- Se comparó la salida obtenida con la esperada y se marcó cada caso como **exitoso** (OK) o **fallido** (FAIL). Los casos que excedían el tiempo límite se marcaron como fallidos por posible ciclo infinito.
- El log resultante incluye un resumen de todos los casos y el detalle (entrada, salida esperada y salida obtenida) de hasta tres casos fallidos y uno exitoso de referencia.

Así, el log cumple el papel que tiene en un incidente real: muestra que algo falla y con qué datos, pero no dice dónde está el defecto. El siguiente recuadro muestra un extracto.

```
=== Log de ejecución: abc233_c ===
Casos ejecutados: 22 | Exitosos: 21 | Fallidos: 1
  [OK] 000.txt  …  [OK] example2.txt
  [FAIL] killer001.txt
[FALLIDO] killer001.txt — entrada: 4 999999998000000001 … | esperado: 936 | obtenido: 0
[EXITOSO] 000.txt (caso de referencia) — entrada: 4 200376420520689664 … | salida: 0
```

Sistemas y condiciones. Se compararon cuatro sistemas (Tabla 7). Todos recibieron lo mismo: solo el archivo defectuoso en un directorio vacío, el mismo log y la misma instrucción, con herramientas limitadas a lectura. En Gemini CLI se bloqueó la búsqueda web. El cuarto sistema, Gemini API, es el mismo modelo de Gemini CLI llamado directamente, sin herramientas ni razonamiento extendido; se agregó para medir cuánto aporta el andamiaje del agente con el modelo fijo.

Sistema	Modelo	Andamiaje
BugWizard	Claude Sonnet 5 (Bedrock)	RAG, una llamada; razonamiento adaptativo del modelo activo
Claude Code	Claude Sonnet 5 (Bedrock)	Agente con herramientas de solo lectura
Gemini CLI	gemini-3.5-flash	Agente completo con razonamiento activo
Gemini API	gemini-3.5-flash	Sin agente: una llamada, sin herramientas ni razonamiento extendido

Tabla 7. Sistemas evaluados en ConDefects (ejecutados entre julio y septiembre de 2026).

Respuesta en texto, no en número de línea. Se pidió a cada sistema terminar con las tres líneas más probables, copiando su **texto exacto**; el evaluador las busca en el archivo para obtener el número. Las primeras pruebas pedían el número de línea, y eso castigaba a BugWizard de forma injusta: al trabajar con

fragmentos recuperados no conserva la numeración del archivo, y sus diagnósticos eran correctos, pero con el número corrido a la línea vecina. Con el texto, la métrica mide si el sistema encontró el defecto y no si supo contar líneas.

4.3.2. Resultados

Sistema	Top-1	Top-3 / Top-5	Fácil (14)	Medio (8)	Difícil (14)
BugWizard	77,8%	83,3%	85,7%	87,5%	64,3%
Claude Code	69,4%	75,0%	85,7%	75,0%	50,0%
Gemini CLI	**83,3%**	**86,1%**	85,7%	100,0%	71,4%
Gemini API	66,7%	72,2%	85,7%	75,0%	42,9%

Tabla 8. Localización de fallas en ConDefects (n=36). Las columnas de dificultad muestran el Top-1 por estrato.

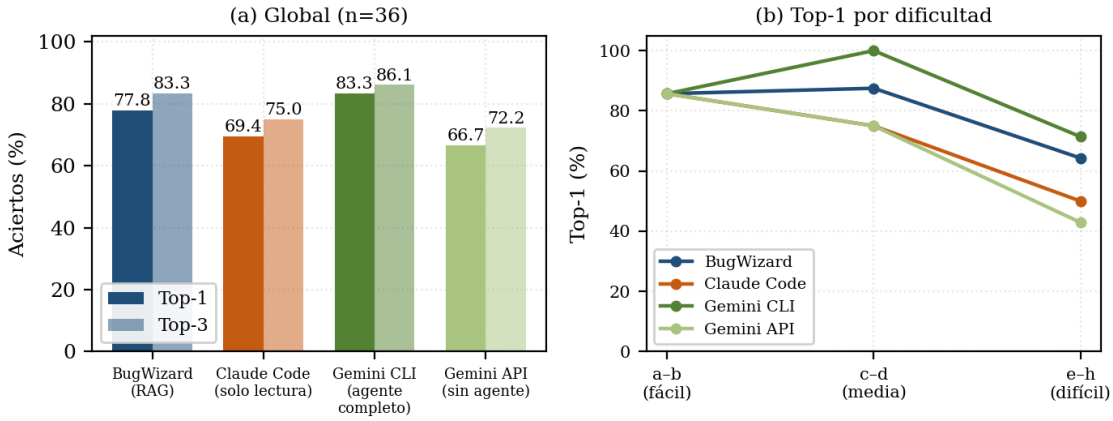


Figura 4. Resultados en ConDefects: (a) aciertos globales; (b) Top-1 según dificultad.

Gemini CLI acertó en la primera línea propuesta en 83% de los programas, BugWizard en 78%, Claude Code en 69% y Gemini API en 67% (Tabla 8). Con 36 programas, diferencias de pocos puntos equivalen a dos o tres programas, por lo que el orden entre los tres primeros debe leerse como tendencia.

En los programas fáciles todos empatan (86%): el defecto es evidente y no hay margen para diferencias. **Las diferencias aparecen en los difíciles**, donde los sistemas van de 43% a 71%.

El andamiaje importa. Gemini CLI y Gemini API usan el mismo modelo; solo cambia que el primero puede razonar y usar herramientas. Esa diferencia vale 17 puntos en el total (83% contra 67%) y 28 puntos en los programas difíciles (71% contra 43%). Es la única comparación de esta fase en la que el modelo es idéntico y solo cambia el andamiaje, por lo que es la que permite una lectura causal.

BugWizard se ubica entre los agentes comerciales. Con una sola llamada y sin herramientas, superó a Claude Code, que usa el mismo modelo y puede leer archivos, y quedó a dos programas de Gemini CLI. En los programas difíciles mantuvo 64%, por encima de Claude Code (50%).

Por último, 23 de los 36 programas fueron resueltos por los tres agentes (BugWizard, Claude Code y Gemini CLI) y 5 no fueron resueltos por ninguno. Es decir, en 28 de 36 casos (78%) los sistemas se comportan igual; las diferencias se concentran en los 8 restantes.

4.3.3. Observaciones de la Fase 2

En pocas palabras. Sobre un benchmark público y con respuesta objetiva, BugWizard acertó la línea defectuosa en 78% de los programas, entre Gemini CLI (83%) y Claude Code (69%). El andamiaje del agente explicó 17 puntos con el modelo fijo, sobre todo en los programas difíciles.

Esta fase tiene un límite que conviene recordar: ConDefects son programas de concurso de un solo archivo, no sistemas distribuidos, y el log construido es más simple que los de producción. Valida la capacidad de los sistemas para encontrar defectos en el código, pero no reemplaza a la Fase 1 en la pregunta sobre el valor de los logs en sistemas reales.

4.4. Fase 3: experimento con desarrolladores

4.4.1. Diseño del experimento

Participantes. Cuatro desarrolladores de Apside que no participaron en la construcción ni en la evaluación de BugWizard y que no conocían los escenarios. En este informe se identifican como D1 a D4.

Tickets. Se usaron los 16 escenarios sintéticos de la Fase 1 que tienen código disponible (del 004 al 020, sin el 017). Para cada uno se redactó un ticket como el que llegaría a soporte: el síntoma y el comportamiento esperado, sin la causa raíz. Los escenarios seguían desplegados en AWS y generando logs mediante los *canaries*.

Diseño cruzado. Cada desarrollador resolvió los 16 tickets, unos con BugWizard y otros con su flujo habitual. Se formaron dos grupos con asignaciones opuestas: lo que D1 y D2 resolvieron con BugWizard, D3 y D4 lo resolvieron sin él, y viceversa. Así, cada ticket se resolvió dos veces con BugWizard y dos veces sin él, por personas distintas, y nadie vio el mismo ticket dos veces. Este tipo de diseño controla las diferencias entre personas [29].

Condiciones.

- **Con BugWizard:** el proyecto del escenario ya configurado (repositorio y grupos de logs). BugWizard usó Claude Sonnet 5 en su modo RAG de una llamada.
- **Flujo habitual:** el desarrollador trabajó como lo haría normalmente, con acceso al repositorio y a la consola de AWS (CloudWatch, DynamoDB, etc.) y libertad para usar asistentes de IA genéricos, como Claude. Así se compara BugWizard contra la práctica real, no contra un desarrollador sin ayuda.

Medición. El cronómetro partía al leer el ticket y se detenía cuando el desarrollador declaraba la causa raíz, que debía escribir en dos o tres líneas junto con el archivo afectado. Cada respuesta se comparó con la causa documentada en la especificación del escenario y se clasificó como correcta, parcial o incorrecta. Una ejecución se consideró **no válida** cuando no se pudo completar por un problema del ambiente (por ejemplo, el proyecto no estaba cargado, el stack no estaba desplegado o no había logs). Todas las sesiones se realizaron en una misma jornada.

4.4.2. Resultados

Se registraron 64 ejecuciones (16 tickets × 4 desarrolladores), 32 por condición. Fueron válidas 22 con BugWizard y 29 con el flujo habitual (Tabla 9).

	Flujo habitual	Con BugWizard
Ejecuciones válidas	29 de 32	22 de 32
Tiempo promedio (min)	12,4	**3,5**
Tiempo mediano (min)	13	**3**
Rango (min)	1 – 21	2 – 7
Causa raíz correcta	14 (48%)	**17 (77%)**
Causa raíz parcial	15 (52%)	5 (23%)
Causa raíz incorrecta	0	0

Tabla 9. Resultados del experimento con desarrolladores (ejecuciones válidas).

Tiempo. Para comparar de forma justa se usaron los 13 tickets que tuvieron al menos una ejecución válida en cada condición, promediando las ejecuciones de cada ticket. **BugWizard fue más rápido en los 13 tickets.** El tiempo promedio bajó de 12,1 a 3,6 minutos, una reducción de **70%**, muy por encima del 30% planteado en H1. Incluso en el escenario menos favorable estimado (Anexo D), la reducción supera el 60%.

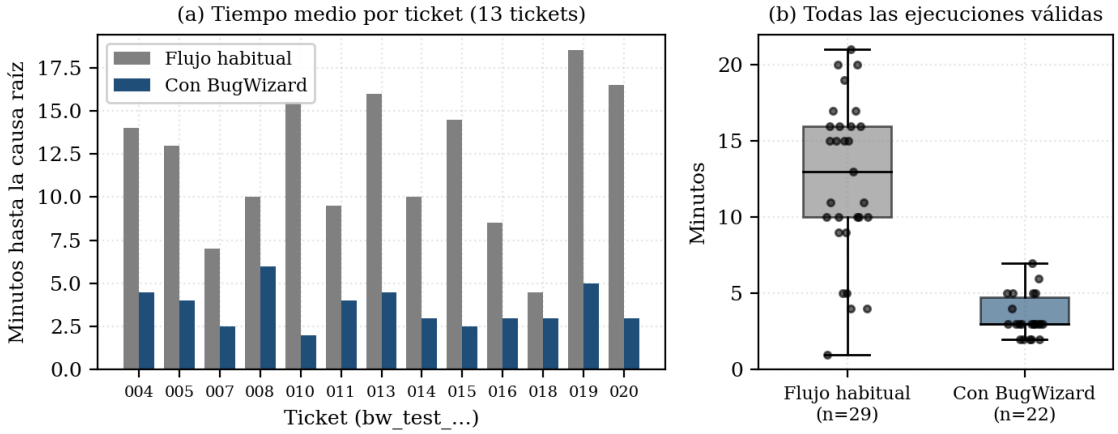


Figura 5. Tiempo hasta identificar la causa raíz: (a) promedio por ticket en cada condición; (b) distribución de todas las ejecuciones válidas.

La reducción fue parecida en todos los niveles de dificultad: 64% en los tickets de dificultad alta, 70% en los de dificultad media y 73% en los de dificultad baja. También fue consistente entre personas: los cuatro desarrolladores fueron más rápidos con BugWizard (Anexo D).

Precisión. Ninguna respuesta fue completamente errada en ninguna condición, pero con BugWizard una mayor proporción llegó a la causa completa (77% contra 48%). Con el flujo habitual era frecuente detenerse en el síntoma visible, por ejemplo "se lanza una excepción" o "hay una condición de carrera", sin llegar al mecanismo de fondo, como la compensación fallida de una transacción o la falta de una operación atómica. La reducción de tiempo, por lo tanto, no se obtuvo a costa de diagnósticos más apresurados.

Ejecuciones no válidas. Con BugWizard hubo 10 ejecuciones no válidas, contra 3 en el flujo habitual. Seis se debieron a problemas del ambiente que afectaban a cualquiera (un escenario sin código cargado, un stack que ya no estaba desplegado). Las otras cuatro ocurrieron con un mismo desarrollador (D4), para quien BugWizard no encontró logs en la ventana de tiempo consultada, aunque otro desarrollador resolvió esos mismos tickets con BugWizard sin problemas. Esto muestra que BugWizard depende de que el proyecto esté bien configurado y de que existan logs para el período del incidente.

4.4.3. Observaciones de la Fase 3

En pocas palabras. Con BugWizard, los desarrolladores identificaron la causa raíz en alrededor de un tercio del tiempo (3,6 contra 12,1 minutos por ticket, −70%) y con más frecuencia llegaron a la causa completa. La ventaja se mantuvo en los 13 tickets comparables, en los tres niveles de dificultad y en los cuatro desarrolladores.

Los comentarios de los participantes explican de dónde sale la diferencia:

- "Es lenta la revisión manual principalmente por el acceso a AWS y encontrar el log adecuado." (D1)
- "El mayor tiempo de los problemas manuales se debió principalmente a buscar el archivo, buscar los logs en CloudWatch (logueo en AWS, doble factor); aun usando IA esto demoraba." (D2)
- "En casi todas las pruebas manuales se tuvo que usar IA para identificar el problema, ya que no era obvio mirando el código." (D2)
- "La herramienta llegó a la conclusión más rápido que yo [...] y entregó más detalle, pero no fue capaz de darme un resumen en dos líneas." (D4)

Es decir, la mayor parte del tiempo del flujo habitual no se fue en razonar, sino en **reunir el contexto**: entrar a AWS, encontrar el grupo de logs y la ventana correcta, y ubicar el archivo. BugWizard elimina esa etapa porque el proyecto ya tiene asociados el repositorio y los logs. Además, varios desarrolladores usaron asistentes de IA genéricos en el flujo habitual, de modo que la ventaja medida no es "usar IA contra no usarla", sino tener un agente ya conectado a las fuentes del proyecto.

Los tiempos absolutos son cortos, de minutos, porque los escenarios son aplicaciones pequeñas; en un incidente real con más servicios los tiempos serían mayores. Lo que se puede extrapolar es la reducción relativa, no los minutos.

5. Discusión

5.1. Los logs son el factor que más pesa

El hallazgo más sólido del trabajo es que el acceso a evidencia de ejecución es lo que más mejora el diagnóstico, y que su efecto crece cuanto más real es el caso. En sistemas serverless, una parte importante de los incidentes no se origina en el código sino en su relación con el entorno: un permiso que falta, una variable que apunta al recurso equivocado, un evento que se pierde entre servicios. Un agente que solo lee código no puede ver esas causas y, en esos casos, no da un diagnóstico incompleto sino uno equivocado con seguridad: propone corregir un código que está bien.

Esto responde al primer vacío de la sección 2.3: los benchmarks de localización de fallas entregan solo código y los de AIOps solo telemetría. La medición de cuánto agrega la observabilidad sobre una base de solo código sugiere que la evaluación de agentes de diagnóstico debería incluir ambas fuentes.

5.2. Un agente de un equipo pequeño frente a agentes comerciales

La pregunta práctica de este trabajo es si vale la pena que una empresa construya su propio agente. La evidencia de las dos primeras fases apunta a que sí es viable:

- En la Fase 1, BugWizard empató con Claude Code, que usa el mismo modelo y consulta los logs de forma interactiva, y superó a Gemini CLI.
- En la Fase 2, sobre un benchmark público, BugWizard (78%) quedó entre Gemini CLI (83%) y Claude Code (69%).

Lo notable es que lo logró con una arquitectura simple: una sola llamada al modelo, sin herramientas, con el contexto preparado de antemano. Cuando el contexto correcto ya está indexado, buena parte del trabajo que hace un agente iterativo, buscar el archivo y el log adecuados, ya está hecha.

Construir una herramienta propia da además **control** sobre aspectos que importan a una empresa y que un agente genérico no ofrece de la misma forma: el modelo corre dentro de la cuenta corporativa y los datos no salen de ella; el equipo decide qué fuentes se indexan, con qué instrucciones y con qué modelo; las credenciales y la configuración de cada proyecto quedan centralizadas; y cambiar de modelo es un cambio de configuración, como ocurrió entre las Fases 1 y 2 al pasar de Sonnet 4 a Sonnet 5. Para una empresa mediana esto significa poder adoptar IA en el diagnóstico sin depender por completo de una herramienta de terceros.

5.3. El andamiaje también importa

La Fase 2 mostró que, con el mismo modelo, dejar al agente razonar y usar herramientas sube el acierto en 17 puntos, y en 28 puntos en los problemas difíciles. Esto matiza el resultado de Traversal [26], según el cual razonar más no siempre mejora: aquí el razonamiento no aportó nada en los programas fáciles, donde todos empatan, y aportó mucho en los difíciles. Promediar sobre una muestra con muchos casos fáciles esconde un efecto que sí existe donde más importa.

Que BugWizard, sin herramientas, supere a Claude Code con herramientas parece contradecir esto, pero no es así: BugWizard tenía activo el razonamiento adaptativo del modelo, y en ConDefects el contexto cabe completo en un solo archivo, de modo que la ventaja de poder explorar es pequeña. Lo que sí muestra es que comparar sistemas sin controlar el andamiaje produce rankings que reflejan decisiones de configuración más que la capacidad del modelo.

5.4. El costo de reunir el contexto

Antes del experimento con desarrolladores, BugWizard se había probado en el ambiente de un cliente de Apside, fuera de un protocolo formal. Allí se observó que gran parte del tiempo se iba en saber qué repositorio correspondía al servicio afectado, conseguir credenciales para los logs y encontrar la ventana de tiempo, y que con el proyecto ya configurado en BugWizard esa etapa desaparecía.

La Fase 3 confirmó esa observación de forma controlada: los desarrolladores atribuyeron la mayor parte de su tiempo en el flujo habitual a entrar a AWS y encontrar los logs. La implicancia es que el valor de un agente de diagnóstico no está solo en su capacidad de razonar, sino también en **tener ya establecidas las conexiones con las fuentes de información**. La comparación que importa para una empresa no es entre modelos, sino entre un agente configurado para su infraestructura y uno genérico que debe recibir el contexto cada vez.

5.5. Limitaciones

Muestras pequeñas. Hay 19 escenarios, 10 incidentes, 36 programas y 4 desarrolladores. Por eso los resultados deben leerse como evidencia consistente entre fases y no como estimaciones precisas. En la Fase 1, al corregir por el número de comparaciones, ninguna diferencia entre pares alcanza significancia (Anexo B).

Evaluadores. En la Fase 1 hubo dos expertos y no tres como se había previsto, y uno de ellos es el autor; ese experto calificó algo más alto que el otro, aunque ambos coincidieron en el orden. En la Fase 3 la clasificación de las respuestas como correcta o parcial la hizo el autor.

Diseño de la configuración solo código. En los escenarios sintéticos, *solo código* recibió solo una instrucción genérica, sin el síntoma. Por eso su diferencia con *código + logs* mide el efecto conjunto de conocer el síntoma y tener los logs. En los incidentes reales, donde la diferencia es más grande, la conclusión de que sin logs ciertas causas son imposibles de ver se apoya en la naturaleza de esos incidentes (T005, T008, T010) y no depende de esta diferencia.

Modelos distintos entre fases. Los modelos cambiaron entre la Fase 1 (febrero de 2026) y las Fases 2 y 3 (julio a septiembre de 2026), porque los anteriores dejaron de estar disponibles o fueron reemplazados. Por eso las comparaciones que cruzan fases deben hacerse con cuidado. Las comparaciones dentro de cada fase no se ven afectadas, porque en cada una todos los sistemas se ejecutaron en la misma ventana de tiempo.

Representatividad. Los incidentes reales vienen de una sola consultora y de un tipo de arquitectura (serverless en AWS). ConDefects son programas de concurso, distintos del código empresarial. Los escenarios de la Fase 3 son pequeños y fueron diseñados por el autor.

Operación del experimento con desarrolladores. Hubo más ejecuciones no válidas con BugWizard (10) que sin él (3). Si esas ejecuciones se hubieran contado como fallas, el beneficio sería menor; se excluyeron porque se debieron al ambiente y no a la herramienta, pero muestran que en la práctica BugWizard depende de una buena configuración.

6. Conclusiones

6.1. Respecto de las hipótesis

H1: BugWizard, integrado al flujo del desarrollador, reduce el tiempo de diagnóstico (MTTD) en al menos 30% sin empeorar la precisión de la causa raíz.

se sostiene. Con BugWizard, los desarrolladores identificaron la causa raíz 70% más rápido, en los 13 tickets comparables, en los tres niveles de dificultad y en los cuatro participantes, y con más respuestas completas que en su flujo habitual. La reducción supera con margen el 30% planteado. Dado el número de participantes, el resultado debe considerarse exploratorio.

H2: Las configuraciones con acceso a logs producen mejores diagnósticos que las que solo disponen del código, según evaluación ciega de expertos.

se sostiene. Las configuraciones con logs superaron consistentemente a las de solo código, en ambos conjuntos de casos y según ambos expertos. En incidentes reales, la identificación de la causa raíz pasó de 40% a 100%.

H3: El orden de desempeño entre sistemas observado en el banco interno se mantiene en un benchmark público independiente.

se sostiene en parte. La posición de BugWizard respecto de Claude Code se mantuvo entre el banco interno y ConDefects (empatados en el primero, BugWizard algo por encima en el segundo). La de Gemini, en cambio, se invirtió: último en la Fase 1 y primero en la Fase 2. Entre ambas fases cambiaron a la vez la tarea (diagnóstico con logs contra localización en un solo archivo) y los modelos, así que no es posible saber cuál de los dos cambios explica la inversión.

H4: Un agente construido con RAG por un equipo pequeño (BugWizard) logra un desempeño comparable al de agentes comerciales de proveedores globales (Claude Code, Gemini CLI) en esta tarea.

se sostiene. BugWizard, construido por un equipo pequeño con RAG y una sola llamada al modelo, empató con el mejor agente comercial en la Fase 1 y quedó entre los dos agentes comerciales en la Fase 2. Para esta tarea, una solución propia y simple es una alternativa real a las herramientas de los proveedores globales, con la ventaja adicional del control sobre los datos y la configuración.

6.2. Contribuciones

La primera contribución es la **medición controlada de cuánto mejora el diagnóstico al agregar logs** a una base de solo código, que no se encontró en la literatura revisada.

La segunda es la **evidencia de que un agente propio y simple compite con agentes comerciales** en esta tarea, lo que abre una opción concreta a empresas medianas que quieren más control sobre sus herramientas.

La tercera es la **medición del efecto sobre el trabajo de los desarrolladores**, que muestra que gran parte del beneficio viene de tener el contexto ya conectado y no solo de la capacidad del modelo.

La cuarta es una **guía de adopción** para consultoras de TI medianas:

- Lo que más rinde es conectar el agente a los logs de producción; la diferencia entre modelos es menor que la diferencia entre tener y no tener logs.
- La configuración de los proyectos (repositorios, fuentes de logs, credenciales) es parte de la implementación, no un detalle operativo: de ella depende tanto el ahorro de tiempo como que la herramienta funcione.
- Un agente propio basado en RAG es una alternativa viable a los agentes comerciales, y permite mantener los datos dentro de la empresa.
- Conviene fijar la versión exacta del modelo y prever su retiro: durante este trabajo, varios de los modelos evaluados dejaron de estar disponibles.

En lo metodológico, el trabajo deja dos aprendizajes: los escenarios sintéticos subestiman el valor de los logs, y la forma de medir importa (pedir el número de línea en vez del texto castigaba a un sistema por su arquitectura y no por su capacidad).

6.3. Trabajo futuro

Evaluar la versión actual de BugWizard. Después de las pruebas, BugWizard incorporó un modo de agente con herramientas MCP: un explorador de código en un entorno aislado, un inspector de configuración de AWS y el servidor MCP oficial de AWS. Repetir la Fase 1 con esta versión permitiría comparar en el mismo sistema el modo RAG contra el modo agente. Las pruebas preliminares sugieren que las herramientas hacen las respuestas más verificables, porque citan archivo y línea y dejan un rastro de los pasos, pero no garantizan un mejor diagnóstico en cada ejecución.

Ampliar el experimento con desarrolladores, con más participantes, incidentes más grandes y una medición separada del tiempo para reunir el contexto y del tiempo de análisis, para confirmar de dónde viene el ahorro.

Separar el efecto del síntoma y de los logs en la Fase 1, agregando una configuración de código con síntoma y sin logs.

Validar en un benchmark público con telemetría y código, y extender el estudio a otras arquitecturas, como contenedores orquestados o monolitos, para ver si los resultados se mantienen fuera del entorno serverless.

Referencias

[1] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[2] T. Brown et al., "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.

[3] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824–24837.

[4] L. Wang et al., "A Survey on Large Language Model based Autonomous Agents," *Frontiers of Computer Science*, vol. 18, no. 6, art. 186345, 2024.

[5] L. Weng, "LLM Powered Autonomous Agents," *Lil'Log*, jun. 2023. [En línea]. Disponible: https://lilianweng.github.io/posts/2023-06-23-agent/

[6] Anthropic, "Building Effective AI Agents," *Anthropic Research*, dic. 2024. [En línea]. Disponible: https://www.anthropic.com/research/building-effective-agents

[7] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. International Conference on Learning Representations (ICLR)*, 2023.

[8] Anthropic, "Claude Code overview," *Claude Code Docs*, 2025. [En línea]. Disponible: https://code.claude.com/docs

[9] Google, "Gemini CLI: An open-source AI agent that brings the power of Gemini directly into your terminal," repositorio GitHub, 2025. [En línea]. Disponible: https://github.com/google-gemini/gemini-cli

[10] C. E. Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," in *Proc. International Conference on Learning Representations (ICLR)*, 2024.

[11] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[12] J. Johnson, M. Douze y H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[13] P. Toro Isaza et al., "Retrieval Augmented Generation-Based Incident Resolution Recommendation System for IT Support," *arXiv preprint* arXiv:2409.13707, sep. 2024.

[14] Anthropic, "Introducing the Model Context Protocol," *Anthropic News*, nov. 2024. [En línea]. Disponible: https://www.anthropic.com/news/model-context-protocol

[15] Y. Dang, Q. Lin y P. Huang, "AIOps: Real-World Challenges and Research Innovations," in *Proc. IEEE/ACM 41st International Conference on Software Engineering: Companion (ICSE-Companion)*, 2019, pp. 4–5.

[16] T. Wang y G. Qi, "A Comprehensive Survey on Root Cause Analysis in (Micro) Services: Methodologies, Challenges, and Trends," *arXiv preprint* arXiv:2408.00803, jul. 2024.

[17] W. E. Wong, R. Gao, Y. Li, R. Abreu y F. Wotawa, "A Survey on Software Fault Localization," *IEEE Transactions on Software Engineering*, vol. 42, no. 8, pp. 707–740, 2016.

[18] B. Beyer, C. Jones, J. Petoff y N. R. Murphy, *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, EE. UU.: O'Reilly Media, 2016.

[19] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang y S. Rajmohan, "Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models," in *Proc. 45th International Conference on Software Engineering (ICSE)*, 2023.

[20] Meta Engineering, "Leveraging AI for efficient incident response," *Meta Engineering Blog*, jun. 2024. [En línea]. Disponible: https://engineering.fb.com/2024/06/24/data-infrastructure/leveraging-ai-for-efficient-incident-response/

[21] M. Ma et al., "Optimizing incident management with AIOps using the Triangle System," *Microsoft Azure Blog*, mar. 2025. [En línea]. Disponible: https://azure.microsoft.com/en-us/blog/optimizing-incident-management-with-aiops-using-the-triangle-system/

[22] D. Roy et al., "Exploring LLM-based Agents for Root Cause Analysis," *arXiv preprint* arXiv:2403.04123, mar. 2024.

[23] Y. Chen et al., "Automatic Root Cause Analysis via Large Language Models for Cloud Incidents," in *Proc. 19th European Conference on Computer Systems (EuroSys)*, Atenas, Grecia, abr. 2024.

[24] W. Xu et al., "LogSage: An LLM-Based Framework for CI/CD Failure Detection and Remediation with Industrial Validation," *arXiv preprint* arXiv:2506.03691, 2025.

[25] J. Xu et al., "OpenRCA: Can Large Language Models Locate the Root Cause of Software Failures?," in *Proc. International Conference on Learning Representations (ICLR)*, 2025. [En línea]. Disponible: https://openreview.net/forum?id=M4qNIzQYpd

[26] Traversal Research, "LLM Benchmarking for In-Context Retrieval and Reasoning in Incident Root Cause Analysis," *Traversal Research & Insights*, 2025. [En línea]. Disponible: https://traversal.com/blog/llm-benchmarking-in-context-retrieval-reasoning-incident-root-cause-analysis

[27] Y. Wu, Z. Li, J. M. Zhang y Y. Liu, "ConDefects: A Complementary Dataset to Address the Data Leakage Concern for LLM-Based Fault Localization and Program Repair," *arXiv preprint* arXiv:2310.16253, 2023. [En línea]. Disponible: https://github.com/appmlk/ConDefects

[28] AWS Labs, "Log Analyzer with MCP," repositorio GitHub, 2025. [En línea]. Disponible: https://github.com/awslabs/Log-Analyzer-with-MCP

[29] S. Vegas, C. Apa y N. Juristo, "Crossover Designs in Software Engineering Experiments: Benefits and Perils," *IEEE Transactions on Software Engineering*, vol. 42, no. 2, pp. 120–135, 2016.

[30] M. Friedman, "The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance," *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 675–701, 1937.

[31] F. Wilcoxon, "Individual Comparisons by Ranking Methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[32] K. Krippendorff, *Content Analysis: An Introduction to Its Methodology*, 2.ª ed. Thousand Oaks, CA, EE. UU.: Sage, 2004.

[33] B. Efron y R. J. Tibshirani, *An Introduction to the Bootstrap*. Nueva York, NY, EE. UU.: Chapman & Hall, 1993.

Anexos

Anexo A. Rúbrica de evaluación de la Fase 1

Dimensión	Escala	Criterio
Puntuación general	1–5	Utilidad global del diagnóstico para un ingeniero que enfrenta el incidente.
Calidad de la explicación	1–5	Claridad, precisión técnica y relación del razonamiento con la evidencia citada.
Calidad de las propuestas	1–5	Qué tan específicas y aplicables son las correcciones sugeridas.
Ausencia de alucinación	1–5	5 = ninguna afirmación sin respaldo; 1 = afirmaciones centrales sin respaldo en el material.
Identificó la causa raíz	Sí / No	Coincidencia con la causa raíz documentada en la resolución del incidente.

Tabla A.1. Rúbrica aplicada por los dos expertos.

Anexo B. Análisis estadístico de la Fase 1

Se usaron pruebas no paramétricas, adecuadas para escalas ordinales y muestras pequeñas: la prueba de Friedman para saber si la configuración influye en cada dimensión [30], la prueba de rangos con signo de Wilcoxon para comparar pares de configuraciones [31], con corrección de Bonferroni, y el α de Krippendorff, la correlación de Spearman y porcentajes de acuerdo para medir la concordancia entre expertos [32].

Dimensión	χ^2 de Friedman	p	¿Efecto significativo?
Calidad de la explicación	54,93	<0,001	Sí
Puntuación general	36,27	<0,001	Sí
Calidad de las propuestas	33,23	<0,001	Sí
Nivel de alucinación	10,91	0,149	No
Identificó el error	10,17	0,199	No

Tabla B.1. Prueba de Friedman sobre los escenarios sintéticos (n=19, ocho configuraciones).

En las comparaciones por pares, ningún par supera la corrección de Bonferroni (α=0,0018), consecuencia de comparar 28 pares con n=19. Sin corrección, doce pares tienen p<0,05 (Tabla B.2), y todos oponen una configuración con logs a una sin logs o a Gemini. Ninguna comparación entre BugWizard y Claude Code con logs es significativa; por ejemplo, BugWizard (prompt 1) contra Claude Code (detallado) da p=0,681.

Configuración A	Configuración B	p	Mejor
Claude Code (código + logs)	Gemini CLI (solo código)	0,0087	A
Claude Code (solo código)	Claude Code (detallado)	0,0113	B
Claude Code (detallado)	Gemini CLI (solo código)	0,0113	A
BugWizard (prompt 1)	Claude Code (solo código)	0,0147	A
Claude Code (solo código)	Claude Code (código + logs)	0,0172	B
Claude Code (detallado)	Gemini CLI (código + logs)	0,0183	A
BugWizard (prompt 1)	Gemini CLI (solo código)	0,0224	A
BugWizard (prompt 2)	Gemini CLI (solo código)	0,0239	A
BugWizard (prompt 2)	Gemini CLI (detallado)	0,0269	A
Claude Code (detallado)	Gemini CLI (detallado)	0,0355	A
BugWizard (prompt 1)	Gemini CLI (código + logs)	0,0414	A
BugWizard (prompt 2)	Claude Code (solo código)	0,0492	A

Tabla B.2. Comparaciones de Wilcoxon con p<0,05 sin corrección (escenarios sintéticos).

El α de Krippendorff fue bajo (entre −0,03 y +0,38), pero esta medida es muy exigente cuando casi todas las notas están en el rango alto, como aquí. Las otras medidas son más informativas: la correlación de Spearman estuvo entre 0,44 y 0,73, el acuerdo dentro de ±1 punto entre 65% y 100%, y ambos expertos produjeron el mismo ranking en los cuatro primeros y los dos últimos lugares. Uno de los expertos calificó sistemáticamente más bajo (promedio 3,76 contra 4,42 en el mejor sistema), sin alterar el orden.

Dimensión	α Krippendorff	ρ Spearman	Acuerdo exacto	Dentro de ±1
Identificó error	+0,379	0,726	81,5%	100,0%
Calidad de la explicación	+0,280	0,539	34,1%	83,7%
Calidad de las propuestas	+0,082	0,441	27,4%	65,2%
Nivel de alucinación	−0,034	0,516	20,7%	79,3%
Puntuación general	+0,065	0,479	27,4%	65,9%

Tabla B.3. Concordancia entre los dos expertos (escenarios sintéticos).

Anexo C. Detalle de la Fase 2 (ConDefects)

Letra del problema	Programas	Estrato
a	10	Fácil
b	4	Fácil
c	4	Medio
d	4	Medio
e	4	Difícil
f	4	Difícil
g	4	Difícil
h	2	Difícil
Total	**36**	

Tabla C.1. Composición del subconjunto de 36 programas Python de ConDefects.

Acuerdo entre sistemas. El porcentaje de programas en que dos sistemas coinciden (ambos aciertan o ambos fallan en Top-1) fue: BugWizard–Gemini CLI 88,9%; BugWizard–Gemini API 88,9%; Claude Code–Gemini API 91,7%; Claude Code–Gemini CLI 86,1%; Gemini CLI–Gemini API 83,3%; BugWizard–Claude Code 80,6%.

Generación de los logs. El script `generate_test_logs.py` ejecuta cada programa defectuoso contra los casos de prueba de su problema (con un límite de 5 segundos por caso), compara la salida con la esperada y escribe un log con el resumen OK/FAIL de todos los casos y el detalle de hasta tres casos fallidos y uno exitoso. Las entradas largas se truncan a 400 caracteres.

Anexo D. Datos del experimento con desarrolladores

Grupo 1 (D1, D2) usó BugWizard en los tickets 004, 006, 007, 011, 014 y 016, y su flujo habitual en los demás; el Grupo 2 (D3, D4), al revés. Minutos hasta declarar la causa raíz; 20 indica ejecución no válida. Entre paréntesis, la clasificación de la respuesta: C = correcta, P = parcial.

Ticket	Dificultad	Habitual (a)	Habitual (b)	BugWizard (a)	BugWizard (b)
004	Alta	17 (C)	11 (C)	5 (C)	4 (C)
005	Media	15 (P)	11 (P)	5 (P)	3 (P)
006	Baja	9 (C)	—	—	—
007	Media	10 (C)	4 (P)	3 (C)	2 (C)
008	Media	15 (P)	5 (P)	6 (C)	—
009	Baja	—	—	—	—
010	Media	16 (C)	15 (C)	2 (C)	2 (C)
011	Alta	10 (C)	9 (P)	5 (C)	3 (P)
012	Baja	21 (P)	15 (P)	—	—
013	Baja	16 (C)	16 (C)	7 (C)	2 (C)
014	Media	10 (C)	10 (P)	3 (C)	3 (C)
015	Baja	19 (C)	10 (C)	3 (C)	2 (C)
016	Baja	16 (P)	1 (P)	3 (C)	3 (C)
018	Baja	5 (P)	4 (P)	3 (P)	—

Ticket	Dificultad	Habitual (a)	Habitual (b)	BugWizard (a)	BugWizard (b)
019	Media	17 (P)	20 (P)	5 (P)	—
020	Baja	20 (C)	13 (C)	3 (C)	—

Tabla D.1. Registro de las 64 ejecuciones. Las columnas (a) y (b) corresponden a los dos desarrolladores que resolvieron el ticket en cada condición.

Desarrollador	Habitual: válidas / promedio	BugWizard: válidas / promedio
D1	9 / 16,0 min	5 / 3,8 min
D2	9 / 12,1 min	5 / 3,0 min
D3	6 / 12,0 min	8 / 4,3 min
D4	5 / 7,0 min	4 / 2,3 min

Tabla D.2. Resultados por desarrollador.

Prueba estadística. Sobre los 13 tickets con ejecuciones válidas en ambas condiciones, la prueba de Wilcoxon de rangos con signo [31] da W=0 (BugWizard más rápido en los 13), p=0,0002, con un tamaño de efecto r=0,88. El intervalo de confianza bootstrap al 95% [33] para la reducción del tiempo promedio es [62%; 76%]. Por ticket, la reducción va de 33% (ticket 018) a 87% (ticket 010), con mediana de 69%.

Dificultad	Tickets	Habitual (min)	BugWizard (min)	Reducción
Alta	2	11,8	4,2	64%
Media	6	12,3	3,8	70%
Baja	5	12,0	3,2	73%
Total	**13**	**12,1**	**3,6**	**70%**

Tabla D.3. Tiempo promedio por ticket según dificultad (13 tickets comparables).

Anexo E. Reproducibilidad

Los datos y scripts que respaldan los resultados están organizados así:

- **Fase 1, escenarios sintéticos:** `oe2_datos_brutos.csv` (288 calificaciones), `oe2_medias_por_agente.csv`, `oe2_wilcoxon.csv` y el script `oe2_analisis.py`. Definición de las configuraciones y prompts: `codigo_evaluado_en_test/tests-bugwizard/llm-testing-scripts/` (`projects.json` y `run-tests.sh` de Claude Code y Gemini CLI).
- **Fase 1, incidentes reales:** `tests_problemas_reales/resultados.md` (160 calificaciones).
- **Fase 2:** `condefects-eval/`: subconjunto (`manifest_subset36.csv`), logs generados (`logs/`), predicciones de cada sistema (`predictions/`, `predictions_claude_code/`, `predictions_gemini_cli/`, `predictions_gemini_api/`) y reportes (`report_bugwizard_36.csv`, `report_claude-code_36.csv`, `report_gemini-cli_36.csv`, `report_gemini_api_36.csv`). Herramientas: `condefects-runner/` (BugWizard), `run_cli_baseline.py` (Claude Code y Gemini CLI), `run_gemini_api_baseline.py`, `generate_test_logs.py` y `score.py`.
- **Fase 3:** protocolo y tickets en `tests_dev_experiment/`; registros de cada desarrollador en `resultados_devs/`.